UNIVERSITÀ DEGLI STUDI DI FIRENZE

Advanced Techniques and Tools for Software Development

PROJECT REPORT

Car Rental Management System

A Java desktop application built with TDD, automated testing, MongoDB, Swing UI, Cucumber BDD, Google Guice, Docker, and GitHub Actions CI.

Field	Details
Student	Mohamed Jama Farah
University	Università degli Studi di Firenze
Academic Year	2025 / 2026
Course	Advanced Techniques and Tools for Software Development
Professor	Lorenzo Bettini
Language	Java 17
Build Tool	Apache Maven 3
GitHub	github.com/mohamedjama-farah/car-rental
DockerHub	hub.docker.com/r/mfarah3/car-rental
Date	March 2026

1. Introduction

This report documents the design, implementation, and testing of a Car Rental Management System built as the assessed project for the course 'Advanced Techniques and Tools for Software Development' at the University of Florence. The project was developed following the guidelines of Professor Lorenzo Bettini, based on his book 'TDD, Build Automation, CI' (Leanpub).

The application is a Java desktop program that allows a user to manage a fleet of cars: adding cars, renting them out to customers, and returning them. Although the domain is simple, the project demonstrates every professional software development technique from the course.

1.1 Project Objectives

1. Apply Test Driven Development (TDD) — write failing test first, then make it pass, then refactor.
2. Achieve high test coverage with unit tests, integration tests, and end-to-end tests.
3. Persist data using MongoDB with automated Docker lifecycle management.
4. Build a Java Swing desktop UI with automated AssertJ Swing end-to-end tests.
5. Implement Behaviour Driven Development (BDD) using Cucumber and Gherkin.
6. Apply Dependency Injection using Google Guice.
7. Automate the full build, test, and deployment pipeline using GitHub Actions CI.
8. Containerise the application using Docker and publish the image to DockerHub.
9. Analyse code quality and coverage with SonarCloud, JaCoCo, and PIT mutation testing.

1.2 Professor Requirements — All Met

✓ Use a real database — MongoDB fully implemented, tested, and automatically managed by Docker.

✓ Provide a desktop UI with end-to-end tests — Java Swing with AssertJ Swing UI tests implemented.

✓ Docker must start automatically during the build locally AND in CI — solved with the fabric8 docker-maven-plugin.

✓ Follow the professor's course material closely — all techniques match the course book.

2. Technologies and Tools

The following table lists every technology and tool used in the project, explaining what it does and why it was selected.

Technology	Purpose
Java 17	Main programming language. Java 17 is the current LTS (Long Term Support) version.
Apache Maven 3	Build automation. Manages all dependencies, compiles code, runs tests, packages the JAR.
JUnit 5 (Jupiter)	Testing framework. @Test, @BeforeEach, @AfterEach, @BeforeAll, @AfterAll annotations.
AssertJ	Fluent assertion library. assertThat(x).isEqualTo(y), assertThatThrownBy(), hasMessageContaining().
Mockito	Mocking framework. Creates fake repository objects for unit tests so business logic is tested in isolation.
AssertJ Swing	UI testing. Simulates real user: types text, clicks buttons, reads list contents.
Cucumber 7	BDD framework. Tests written in plain English (Gherkin) in .feature files. Given/When/Then.
MongoDB	NoSQL document database. Stores car data persistently in production and integration tests.
MongoDB Java Driver	Connects Java code to MongoDB. Provides MongoClient, MongoCollection, and Document classes.
Google Guice 7	Dependency Injection framework. @Inject annotation on constructors. bind().to() bindings in modules.
Log4j 2	Logging framework. Structured log messages with class name and log level.
Docker	Container platform. Runs MongoDB as a container during tests. Packages the app as an image.
fabric8 docker-maven-plugin	Starts and stops Docker containers automatically during the Maven build lifecycle. Replaces manual docker-compose.
JaCoCo	Code coverage. Measures which lines and branches are executed by tests. HTML report.
PIT (Pitest)	Mutation testing. Introduces bugs into production code and checks if tests detect them.
SonarCloud	Cloud code quality analysis on every push. Bugs, smells, coverage, duplications.
GitHub Actions	CI/CD pipeline. Runs automatically on every push: build, test, SonarCloud, Docker push.
Xvfb	Virtual display server for Linux CI. Swing tests need a screen — CI servers have none, Xvfb creates a virtual one.

Eclipse IDE	Development environment used locally on macOS.
DockerHub	Container image registry. mfarah3/car-rental:latest pushed after every successful CI build.

3. System Architecture

The application follows a layered architecture with clear separation of concerns. Each layer has one responsibility and communicates through well-defined interfaces.

3.1 Application Layers

Layer	Class and Responsibility
Presentation Layer	CarSwingApp.java — Java Swing desktop window. Handles all user interactions. Delegates business decisions to the service layer.
Service / Business Layer	CarRentalManager.java — Contains ALL business logic. Enforces all business rules. The most important class.
Repository Interface	CarRepository.java — Java interface defining the storage contract. Decouples business logic from storage details.
Data Layer (Test)	InMemoryCarRepository.java — HashMap-based storage. No database. Used in tests.
Data Layer (Production)	CarMongoRepository.java — MongoDB-based storage. Used in production and MongoDB IT tests.
DI Configuration	CarRentalModule.java — Guice module. Binds CarRepository to InMemoryCarRepository.
Entry Point	CarRentalApp.java — Main class. Creates Guice injector. Opens Swing window on EDT.

3.2 Domain Entities

Entity	Description
Car.java	One car in the fleet. Fields: id (auto-increment static counter), make, model, available (boolean). setAvailable() is package-private — only car.rental package can call it. resetIdCounter() resets the static counter in @BeforeEach.
Rental.java	One rental transaction. Fields: id (auto-increment), car (full object reference), customerName. Uses the same resetIdCounter() pattern as Car.

3.3 Repository Pattern and SOLID Principles

The Repository pattern separates business logic from data access. CarRentalManager depends only on the CarRepository interface — never on a specific implementation. This is the Dependency Inversion Principle (D in SOLID).

The practical benefit: to switch from in-memory storage to MongoDB, only one line changes in CarRentalModule. CarRentalManager does not change at all.

4. Development Process — TDD

The entire project was developed following the Test Driven Development discipline. TDD follows three phases called Red-Green-Refactor:

Phase	Description
RED	Write a test that describes new behaviour. Run it — it MUST fail because the code does not exist yet.
GREEN	Write the minimum production code to make the test pass. No more than is needed.
REFACTOR	Clean up the code without changing behaviour. The tests protect you — if they stay green, behaviour is preserved.

4.1 Development Stages — 19 Versions

The project was built in 19 iterative versions, each adding one new technique. Every version kept all previous tests passing.

Version	What Was Added
V1 – V3	Car entity, Rental entity, CarRentalManager with ArrayList. Basic JUnit 5 tests.
V4 – V5	Complete test suite for all business rules. assertThatThrownBy() for exception tests.
V6 – V7	Log4j 2 logging. JaCoCo code coverage. PIT mutation testing.
V8 – V9	CarRepository interface. Mockito mocks. @Mock, @InjectMocks, when(), verify().
V10	GitHub Actions CI pipeline. Build runs automatically on every push.
V11	Dockerfile. Docker image pushed to DockerHub via CI.
V12	InMemoryCarRepository. CarRentalManagerIT integration tests.
V13	SonarCloud code quality analysis integrated with CI.
V14 – V15	MongoDB. CarMongoRepository. CarMongoRepositoryIT with Docker Compose.
V16	Java Swing desktop UI. AssertJ Swing end-to-end UI tests. setName() on every widget.
V17	Cucumber BDD. car-rental.feature file. CarRentalSteps. CucumberIT runner.
V18	Google Guice DI. CarRentalModule. CarRentalApp. @Inject on CarRentalManager.
V19 (final)	Replaced Docker Compose with fabric8 docker-maven-plugin. Fully automatic MongoDB. CI simplified.

5. Key Change — Automatic Docker with fabric8

The most important change from the initial implementation to the final version is the replacement of manual Docker Compose with the fabric8 docker-maven-plugin. This was a direct requirement from Professor Bettini: Docker must start automatically during the build, both locally and in CI.

5.1 Before: Manual Docker Compose

In versions V14–V18, MongoDB required three manual steps:

Step	Command
Step 1 — manual	docker-compose up -d (start MongoDB before tests)
Step 2 — manual	mvn verify (run tests — MongoDB must already be running)
Step 3 — manual	docker-compose down (stop MongoDB after tests)

⚠ **Problem: forgetting step 1 caused all MongoDB integration tests to fail. In GitHub Actions, a separate services: mongodb block was needed in the YAML.**

5.2 After: fabric8 Automatic Docker

With the fabric8 plugin, a developer needs only one command. Maven manages MongoDB automatically as part of the build lifecycle.

Step	Action
Only step 1	Open Docker Desktop on Mac (Docker engine must be running)
Only step 2	mvn verify — Maven starts MongoDB, runs all tests, stops MongoDB

5.3 How fabric8 Works

The fabric8 plugin is configured in pom.xml with two executions that hook into the Maven build lifecycle:

Maven Phase	What fabric8 Does
pre-integration-test	fabric8 starts a MongoDB container (mongo:6.0) on port 27017. It waits until MongoDB logs 'Waiting for connections' — this guarantees MongoDB is truly ready before any test runs. Maximum wait: 30 seconds.

post-integration-test	fabric8 stops and removes the MongoDB container. Cleanup is automatic, even if tests fail.

5.4 Changes to GitHub Actions CI

Change	Explanation
REMOVED	services: mongodb block — no longer needed. fabric8 handles MongoDB.
REMOVED	Manual nc wait loop that checked if MongoDB port was open.
UNCHANGED	xvfb-run mvn verify — MongoDB now starts inside this command automatically.
UPDATED	Action versions: checkout@v4, setup-java@v4, docker actions @v3/@v5.

✓ Final result: a single command — mvn verify — handles compile, unit tests, start MongoDB, integration tests, UI tests, Cucumber tests, stop MongoDB, and generate coverage.

6. Testing Strategy

The project uses a comprehensive multi-layer testing strategy. Each layer is tested independently and together with the layers it depends on.

Test Type	Description
Unit Tests (*Test.java)	Surefire — mvn test. Repository mocked with Mockito. Tests CarRentalManager business logic in isolation. Fast — no database, no Docker.
Integration Tests (*IT.java)	Failsafe — mvn verify. Real InMemoryCarRepository. Tests manager and repository cooperating together.
MongoDB IT Tests (*IT.java)	Failsafe — mvn verify. Real MongoDB started by fabric8. Tests CarMongoRepository against real database.
UI End-to-End Tests (*IT.java)	Failsafe — mvn verify. AssertJ Swing simulates real user. Creates real Swing window, types, clicks, reads results.
Cucumber BDD Tests (*IT.java)	Failsafe — mvn verify. Plain English scenarios in .feature file. Step definitions connect sentences to Java.

6.1 Unit Tests — CarRentalManagerTest

Unit tests isolate CarRentalManager from all external dependencies using Mockito mocks. The repository is fake — it records calls but does nothing real unless stubbed.

Element	Purpose
@Mock CarRepository repository	Creates a fake CarRepository. All methods return null unless stubbed.
@InjectMocks CarRentalManager manager	Creates a REAL CarRentalManager. Injects the fake repository.
when(repo.findById(1)).thenReturn(car)	Stubbing: tells the mock what to return for a specific input.
verify(repo).save(car)	Asserts save() was called. Verifies the manager persists state changes.
assertThatThrownBy(() -> ...)	Verifies correct exception type and message are thrown.

6.2 MongoDB Integration Tests — CarMongoRepositoryIT

Tests run against real MongoDB, started automatically by fabric8. The collection is dropped before each test for complete isolation.

Lifecycle	Purpose
@BeforeAll — connect	Creates one MongoClient shared across all tests in the class.
@BeforeEach — drop + reset	Drops the cars collection. Resets Car.resetIdCounter(). Every test starts with an empty database.
@AfterAll — close	Closes the MongoClient. Runs even if tests fail — prevents connection leaks.

6.3 UI End-to-End Tests — CarSwingAppIT

These tests simulate a real user. AssertJ Swing controls the application programmatically — typing, clicking, and reading the results from the list and labels.

Element	Purpose
GuiActionRunner.execute()	Creates Swing window on the Event Dispatch Thread. Swing is not thread-safe — this is required.
new FrameFixture(app)	Wraps the window. Provides window.textBox(), window.button(), window.list(), window.label().
window.cleanUp() @AfterEach	Closes window and releases resources. Without this, tests fail randomly after the first test.
setName() on every widget	Every JTextField, JButton, JList, and JLabel must have a unique name. AssertJ Swing finds widgets by this name.

6.4 Cucumber BDD — car-rental.feature

The feature file describes application behaviour in plain English. Anyone can read it and understand what the application does. Step definitions in CarRentalSteps.java connect each sentence to Java code.

Element	Description
Scenario: Add a new car	Given the manager is empty / When I add a car / Then the list has 1 car.
Scenario: Rent a car	Given there is a car / When I rent the car / Then the car is unavailable.
Scenario: Return a car	Given there is a rented car / When I return the car / Then the car is available.
{string} and {int}	Parameter expressions. Cucumber extracts values from the feature file and passes them to Java methods.

7. Dependency Injection — Google Guice

Google Guice is a Dependency Injection (DI) framework. Instead of creating objects manually with new, Guice reads the module configuration and creates and wires objects automatically.

Element	What it Does
@Inject on CarRentalManager constructor	Tells Guice: use this constructor. Guice will provide the CarRepository argument automatically.
CarRentalModule.configure()	bind(CarRepository.class).to(InMemoryCarRepository.class) — defines which implementation to inject.
Guice.createInjector(new CarRentalModule())	Creates the injector — the factory that builds all objects according to the bindings.
injector.getInstance(CarRentalManager.class)	Guice creates CarRentalManager: sees @Inject, creates InMemoryCarRepository, injects it. One call.

In unit tests, Guice is not used at all. The Mockito mock is passed directly to the constructor: new CarRentalManager(mockRepository). This is why constructor injection (not field injection) is used — it makes tests simpler and ensures the object is always in a valid state.

8. CI/CD Pipeline — GitHub Actions

Every push to the main branch on GitHub automatically triggers the CI/CD pipeline defined in .github/workflows/maven.yml. The pipeline runs on a fresh Ubuntu Linux virtual machine in GitHub's cloud.

8.1 Pipeline Jobs

Job	Description
Job 1: build	Runs on every push and pull request. Installs Java 17 and Xvfb. Runs xvfb-run mvn verify — compiles, runs all tests (fabric8 handles MongoDB), generates coverage. Then runs SonarCloud analysis.
Job 2: docker	Runs only on direct pushes if the build job succeeds (needs: build). Builds the JAR, logs into DockerHub, builds and pushes the Docker image as mfarah3/car-rental:latest.

8.2 Key CI Steps

Step	What it Does
actions/checkout@v4	Downloads source code from GitHub onto the build machine.
actions/setup-java@v4	Installs Java 17 (Eclipse Temurin). Caches Maven dependencies between builds.
apt-get install xvfb	Installs virtual display server. Required for Swing UI tests on headless CI servers.
xvfb-run mvn verify	Main build command. Provides virtual display for Swing. fabric8 manages MongoDB inside this.
mvn sonar:sonar	Sends coverage data to SonarCloud. Requires SONAR_TOKEN secret.
mvn package -DskipTests	Builds JAR for Docker. Tests skipped — already ran in the build job.
docker/login-action@v3	Authenticates to DockerHub using DOCKERHUB_USERNAME and DOCKERHUB_TOKEN secrets.
docker/build-push-action@v5	Builds Docker image from Dockerfile and pushes to DockerHub.

8.3 GitHub Secrets

Secret	Purpose
SONAR_TOKEN	Authentication token for SonarCloud.
DOCKERHUB_USERNAME	DockerHub username (mfarah3).
DOCKERHUB_TOKEN	DockerHub access token for pushing images.

9. Code Quality Tools

9.1 JaCoCo — Code Coverage

JaCoCo instruments bytecode before tests run and measures which lines and branches are executed. It generates an HTML report in target/site/jacoco/. To view it locally:

```
open -a Safari target/site/jacoco/index.html
```

9.2 PIT — Mutation Testing

PIT (Pitest) goes beyond line coverage by actually introducing bugs (mutations) into production code and checking whether tests detect them. A 'killed' mutation means a test caught the bug — good. A 'survived' mutation means the test suite has a gap.

Result	Meaning
Killed mutation	A test FAILED when the bug was introduced. The test suite is strong enough for this case.
Survived mutation	All tests PASSED even with a bug. A test is missing or too weak.

```
mvn test-compile org.pitest:pitest-maven:mutationCoverage
```

9.3 SonarCloud — Continuous Quality Analysis

SonarCloud analyses every push to GitHub. The project is configured with sonar.qualitygate.wait=false so that a quality gate failure does not block the CI pipeline build.

Metric	What SonarCloud Checks
Bugs	Errors that will cause incorrect behaviour.
Code Smells	Maintainability issues: duplication, long methods, poor naming.
Coverage	Percentage covered by tests — fed from JaCoCo data.
Security	Potential vulnerabilities and security hotspots.

10. Build and Run Instructions

10.1 Prerequisites

Requirement	How to Get It
Java 17	adoptium.net — Eclipse Temurin 17
Apache Maven	brew install maven (on Mac)
Docker Desktop	docker.com — must be open before running mvn verify
Git	To clone the repository

10.2 Clone and Build

Step	Command
1 — Clone	git clone https://github.com/mohamedjama-farah/car-rental.git
2 — Enter folder	cd car-rental
3 — Open Docker	Open Docker Desktop. Wait for the whale icon to stop animating.
4 — Build & Test	mvn verify
5 — See coverage	open -a Safari target/site/jacoco/index.html

10.3 Expected Output

```
DOCKER> [mongo:6.0]: Start container ...
```

```
DOCKER> Waiting on log out 'Waiting for connections'
```

```
Tests run: 28, Failures: 0, Errors: 0, Skipped: 0
```

```
DOCKER> [mongo:6.0]: Stop and remove container
```

```
[INFO] BUILD SUCCESS
```

10.4 Common Errors and Solutions

Error	Solution
Cannot connect to Docker daemon	Docker Desktop is not running. Open it and wait for it to start.
Port 27017 already in use	Old container still running. Run: docker stop mongo && docker rm mongo
No POM in this directory	Wrong folder. Run: cd ~/eclipse-workspace/car-rental

Swing tests fail in CI	xvfb-run missing. Add: sudo apt-get install -y xvfb in the YAML before mvn verify.

11. Conclusion

This project successfully implements all the techniques and tools taught in the course 'Advanced Techniques and Tools for Software Development'. Starting from a simple Car entity and building up through 19 iterative versions using strict Test Driven Development, the final application demonstrates professional software engineering practices from first principles.

The most significant technical improvement beyond the basic requirements was the integration of the fabric8 docker-maven-plugin. This change transformed the development workflow from a multi-step manual process — requiring separate docker-compose commands — into a single automated command: mvn verify. This command now handles the complete lifecycle: compile, unit tests, start MongoDB, integration tests, Swing UI tests, Cucumber BDD tests, stop MongoDB, and generate coverage — all automatically.

Every requirement given by Professor Bettini was met: the application uses a real MongoDB database, provides a Java Swing desktop UI with comprehensive end-to-end tests, Docker starts and stops automatically during the build, and all techniques closely follow the course book.

11.1 Summary of All Techniques Implemented

Technique	Implementation Detail
TDD (Red-Green-Refactor)	Every feature built test-first throughout all 19 versions.
Unit Testing + Mockito	@Mock, @InjectMocks, when(), verify(), assertThatThrownBy().
Integration Testing	Real InMemoryCarRepository — no mocks. Tests cooperation between classes.
MongoDB Integration Testing	Real MongoDB via fabric8. Collection dropped before each test.
UI End-to-End Testing	AssertJ Swing. Real Swing window. Simulates typing, clicking, reading.
Cucumber BDD	.feature file in Gherkin. Step definitions. CucumberIT runner.
Repository Pattern	CarRepository interface. Two implementations. SOLID Dependency Inversion.
Dependency Injection	Google Guice. @Inject. CarRentalModule. Injector.
Logging	Log4j 2 in CarRentalManager and Car.
Code Coverage	JaCoCo. HTML report in target/site/jacoco/.
Mutation Testing	PIT. Killed vs survived mutations. Verifies test suite strength.
Docker + fabric8	Automatic MongoDB lifecycle. mongo:6.0 container. pre/post-integration-test phases.
Multi-stage Dockerfile	Build stage with Maven. Run stage with JRE only. Smaller final image.
GitHub Actions CI/CD	Two jobs: build+test, docker push. Runs on every push to main.
SonarCloud	Quality gate. Bugs, smells, coverage, duplications on every push.
Containerisation + DockerHub	Image pushed as mfarah3/car-rental:latest after every successful build.

11.2 Project Links

Resource	Link
GitHub Source Code	github.com/mohamedjama-farah/car-rental
DockerHub Image	hub.docker.com/r/mfarah3/car-rental
SonarCloud	sonarcloud.io — organisation: mohamedjama-farah